



本科生实验报告

实验课程: 操作系统实验

实验名称: 实验 2

专业名称: 计算机科学与技术

学生姓名: 陈安康

学生学号: 22320021

实验地点: 实验中心 D402

实验成绩: _____

报告时间: 2024/3/21

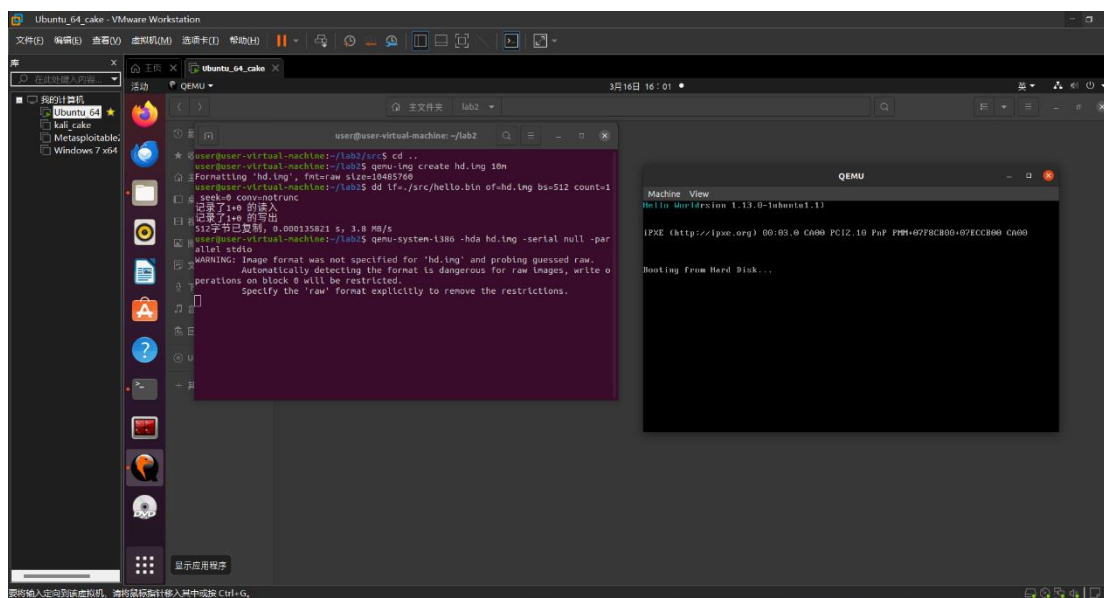
1. 实验要求

学习操作系统引导启动与汇编相关知识，完成四个实验任务要求。

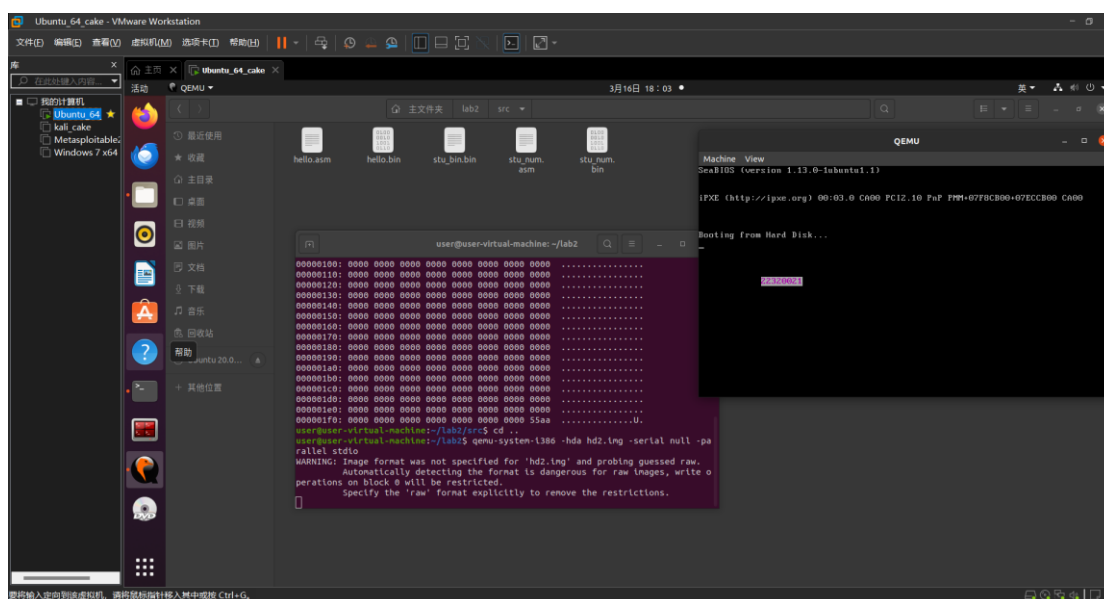
2. 实验过程

任务 1:

按照教程初始化相应的寄存器，然后遵照示例代码完成汇编的编写，创建虚拟磁盘，写入，模拟启动结果如下，代码位于 `hello.asm` 中：



对 `hello` 代码的思路，按照显示原理设置开始输入的单元位置，指定采用的段寄存器为 `gs`，初始地址为 `[gs:2*(80*12+12)]` 依次将 `ah` 与 `al` 设置为需要的颜色和字符，依次输出即可。结果如下图，代码位于 `stu_num.asm` 中：



任务 2:

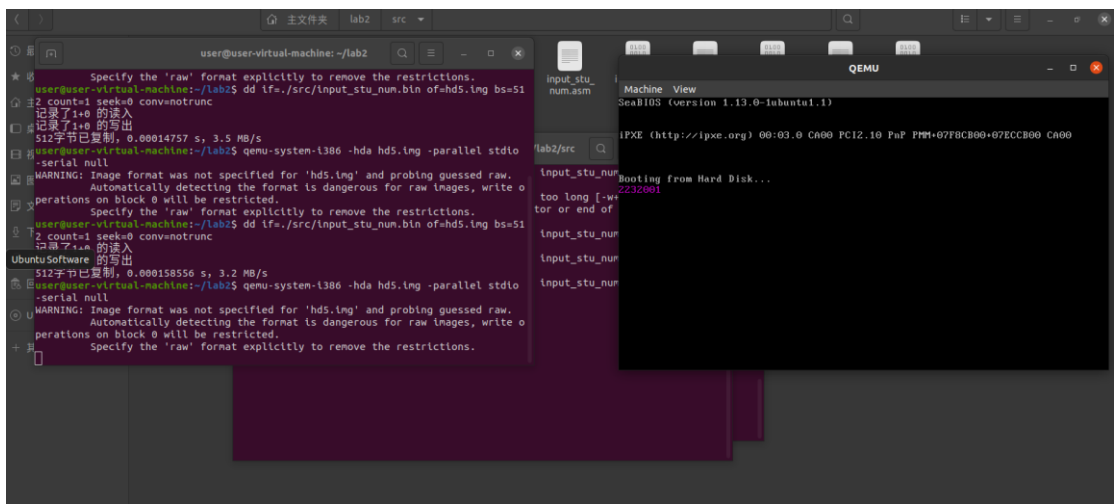
按照教程，学习中断基本流程，先将相关寄存器设置好预定的值，然后执行中断，执行完后从约定好的寄存器取出数值。

在 `get_cursor_pos` 代码中，我先按照中断设置了光标的位置，然后再次执行中断获得光标当前的位置，计划从初始显示位置显示，但这里就有问题了，从 `dh` 与 `d1` 获得的行号和列号都是数值，如果是个位数可以轻易转换成字符，但如果行号和列号是两位数时不能直接获取其对应的字符串了，这里考虑到显存大小是 `80x25`，最多两位数，因此思路是每个坐标信息都统一采用两个字符表示，仅一位数时高位补零。通过坐标信息除以 10 分别获得商和余数，各自转换成字符并显示。这里我刚开始时遇到了一点小问题，开始时没有清除寄存器中的数值，导致开始时显示不了，后来经过检查和学习，采用 “`xor <寄存器>, <寄存器>`” 指令去清除寄

寄存器中的值以保证最后能够成功显示。

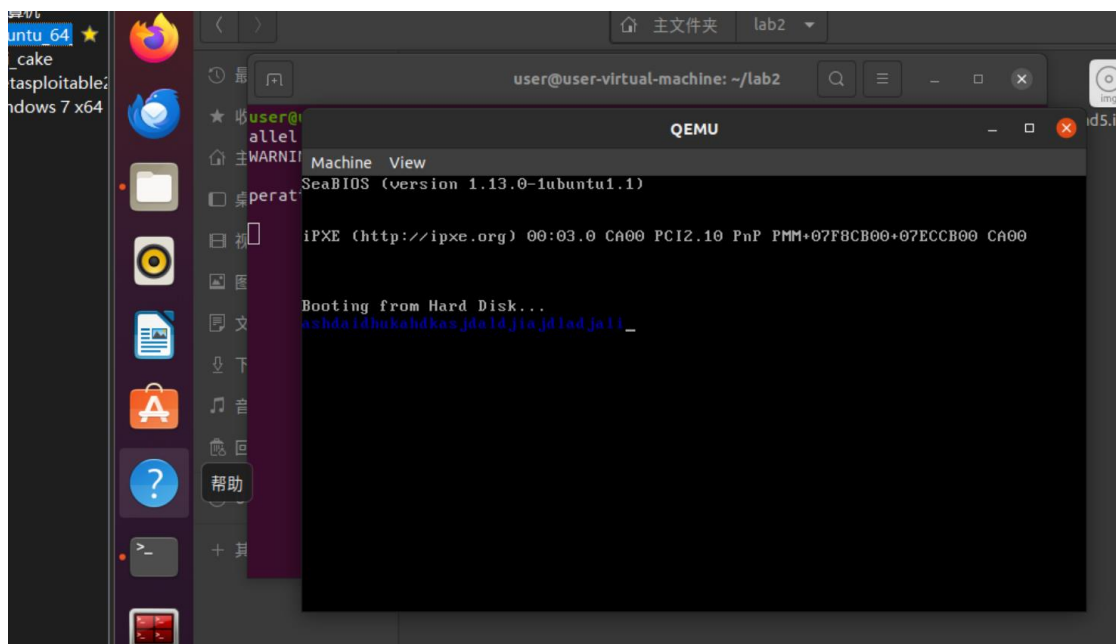


实模式下中断输出学号思路就比较简单了，先采用中断在当前位置输出一位学号，然后获取光标位置存入 `d1` 与 `dh` 寄存器中，接着中断向后移动一位光标，再继续输入一位学号，以此往复，完成学号输入，代码位于 `input_stu_num.asm`。



学习了键盘常用的中断后，选择采用 `16H`（获取键盘输入字符）和 `04H`（关闭键盘输入）配合之前的中断中断来实

现这个效果，先中断获得键盘输入字符并采用先前的中断在当前光标中输出字符，接着就是移动光标的逻辑，获取光标位置，移动光标，只不过这里考虑到换行，所以有一个判断，并且代码会检查输入字符是否为“esc”，如果输入为“esc”则跳入关闭键盘的代码采用 04H 中断关闭键盘，代码位于 keyword.asm 中。



任务 3:

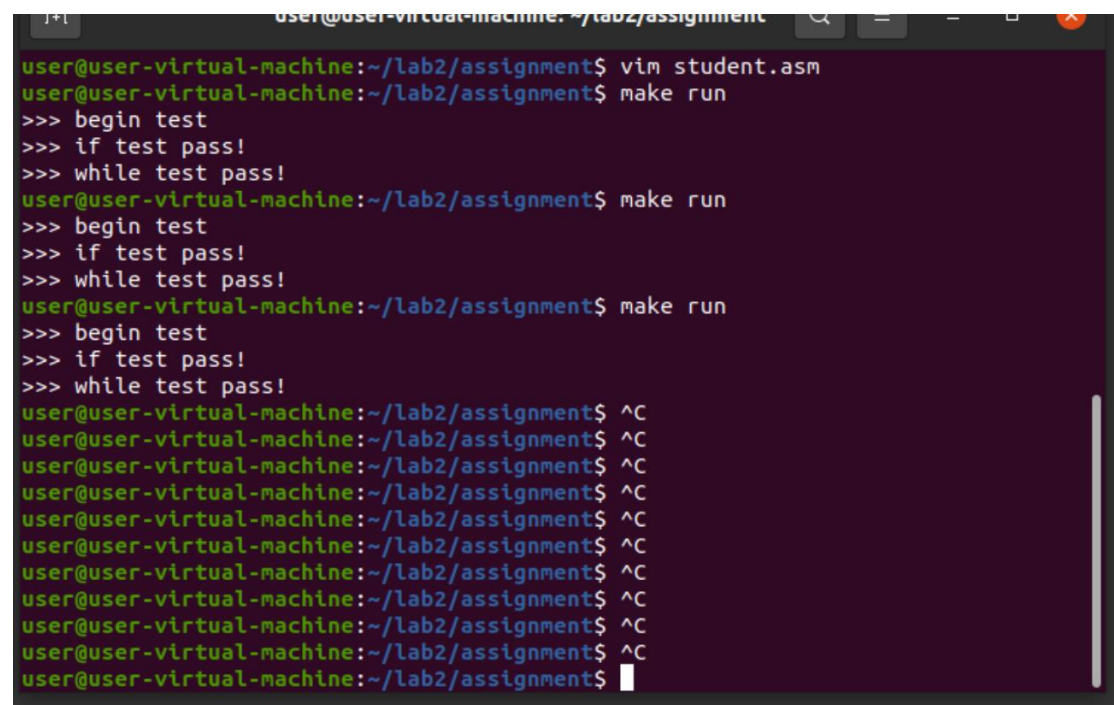
If 语句的汇编实现就是通过“cmp”指令与“je”等跳转指令以及标号设置来完成跳转。然后按照要求写汇编就可以。

While 语句汇编实现通过设置标号与“cmp”指令和“je”等指令来实现控制转移。有一个值得注意的地方是给定的函数不仅会改变 eax 的值，也会改变 edx 的值，这一点是代码运行出错经过 gdb 调试发现的，在调用函数前后采用出入栈

保存 `edx` 寄存器的值以保证运行正确。还有一个值得注意的点是 `while_flag` 由于保存的是字符数组，所以它的地址保存的是数组首字符的地址，在寻址和赋值时需要注意使用的方式。

函数实现通过标号和 `ret` 语句实现执行流的控制，按照要求进行编写就行。

代码位于 `student.asm` 中。

A terminal window showing the execution of an assembly program. The prompt is 'user@user-virtual-machine:~/lab2/assignment\$'. The user enters 'vim student.asm', then 'make run'. The program output is: '>>> begin test', '>>> if test pass!', '>>> while test pass!'. Then the user enters 'make run' again, and the program outputs the same three lines. This sequence is repeated several times, with the user pressing '^C' to interrupt the program. The terminal output shows the program running and then being interrupted by the user pressing the Ctrl+C key, indicated by '^C' characters.

```
user@user-virtual-machine:~/lab2/assignment$ vim student.asm
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$
```

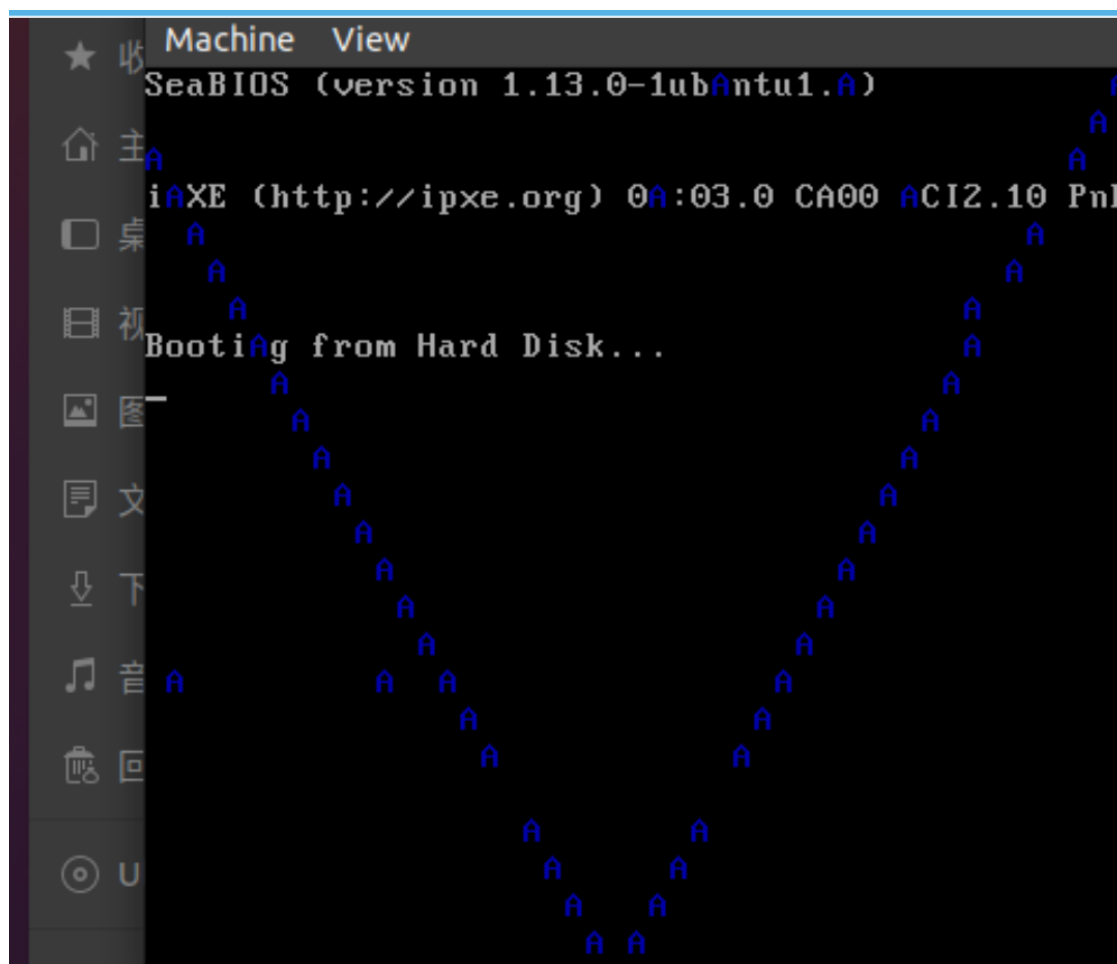
任务 4:

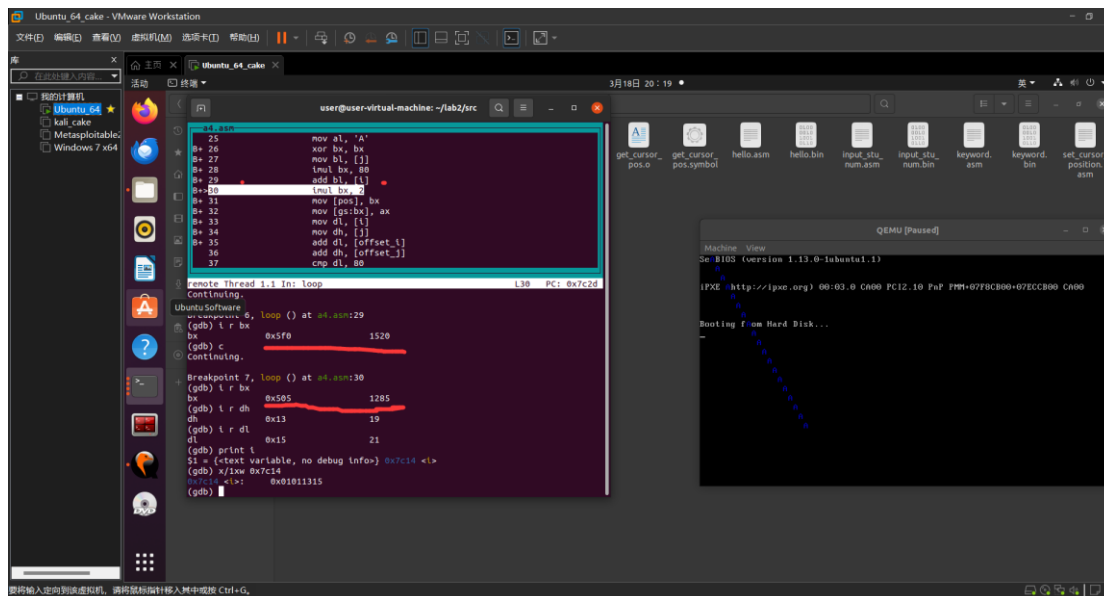
字符弹射程序首先要解决“动画”，即可可视化的先后出现的效果。因此自然想到要有延迟程序，这里在查阅资料后选择通过插入 `nop` 指令来实现延迟，采用计数循环来指定延迟时间。

接着就是字符显示，这里按道理可以有两种字符显示方法，即直接为显存地址赋值和中断，我这里采用了前一种。

接下来最关键的是“移动”，这里我设置了几个数据标号，并且通过对边界条件的判定实现来更改下一步要显示的位置。刚开始时遇到了一些问题如下图：

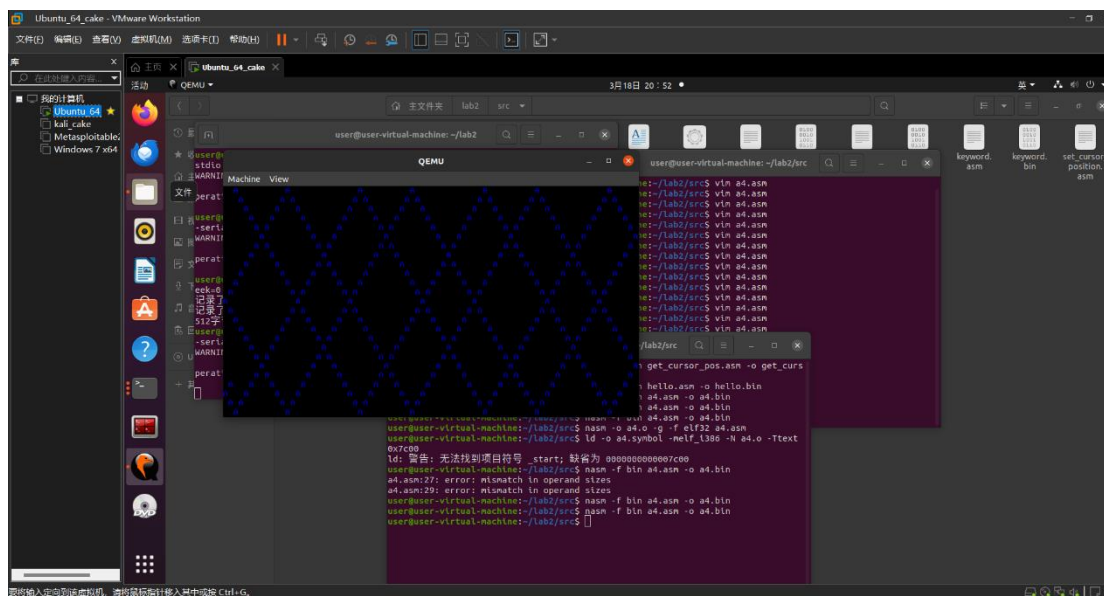
在某些位置会出现突然的跳转（不正常现象如下图），经过 gdb 调试，发现原来是采用的寄存器位数出现了问题，当 d1 与标号中的数据相加产生进位时，由于都是八位数据，进位的 1 默认被抹去了，没有按照我的预想进到 dh 寄存器，所以产生了错误。





（上图所示 bx 经过加法时候本来应该是 0x605，结果是 0x505，是因为上面的加法用的是 b1 寄存器，高位进位自动删除了）

解决方法自然是直接将标号的数据存放到位数更宽的寄存器，但这样直接进行位数不相等的赋值默认是从标号首地址处取出与寄存器位数一致的数据，很可能取到无用值从而影响程序运行，这里我采用赋值后高位清零的方式保证了赋值给寄存器的数据是正确的，从而使程序可以正常运行，代码位于 a4.asm 中。



由于改变字符就是随机 `eax` 寄存器中的数值比较简单（我的思路是用较大数据相乘取余等方法模拟随机数然后通过与或操作控制某些关键位置的数据来模拟随机生成字符，不过没有付诸行动），且时间有点来不及所以没有做这一部分。

3. 关键代码

在 `get_cursor_pos.asm` 中，有代码：

```
xor edx, edx
```

```
xor ecx, ecx
```

```
xor eax, eax
```

这段代码位于除法得余数的前面，用异或清除寄存器中的乱码，保证能够得到正确的结果。

在 `student.asm` 中的 `while` 实现里调用 `my_random` 函数前后：

```
push edx
```

```
call my_random
```

```
pop edx
```

这是因为该函数会改变 `edx` 的值，所以先入栈保存起来，等调用结束之后出栈返回。

在 `a4.asm` 中，标号赋值给寄存器有这样一段代码：

```
xor ax, ax
```

```
mov ax, [i]
```

```
xor ah, ah
```

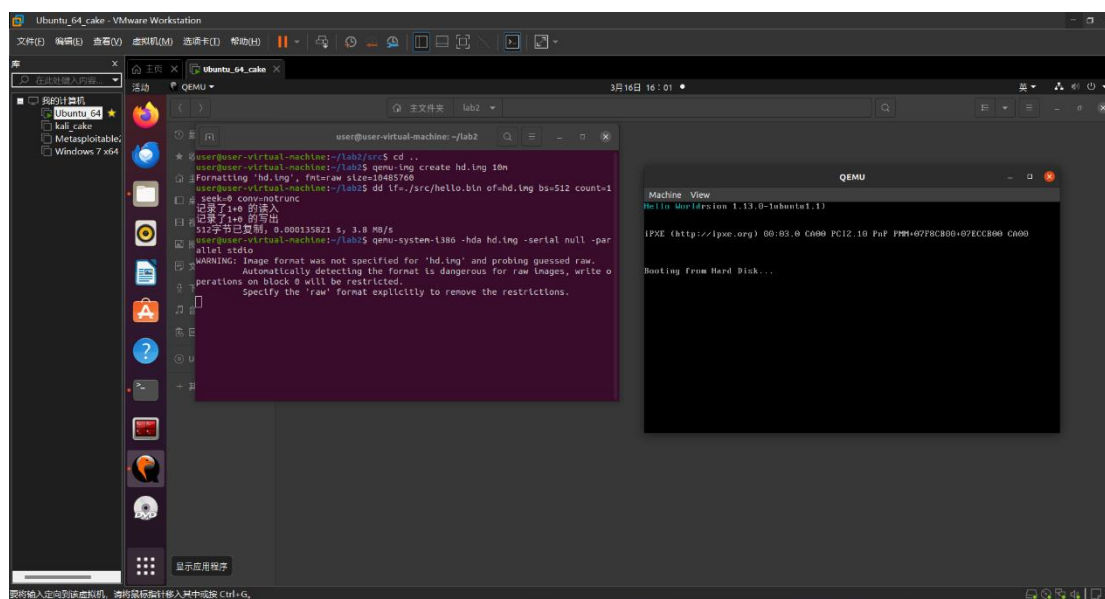
```
xor bx, bx
```

```
mov bx, [j]
```

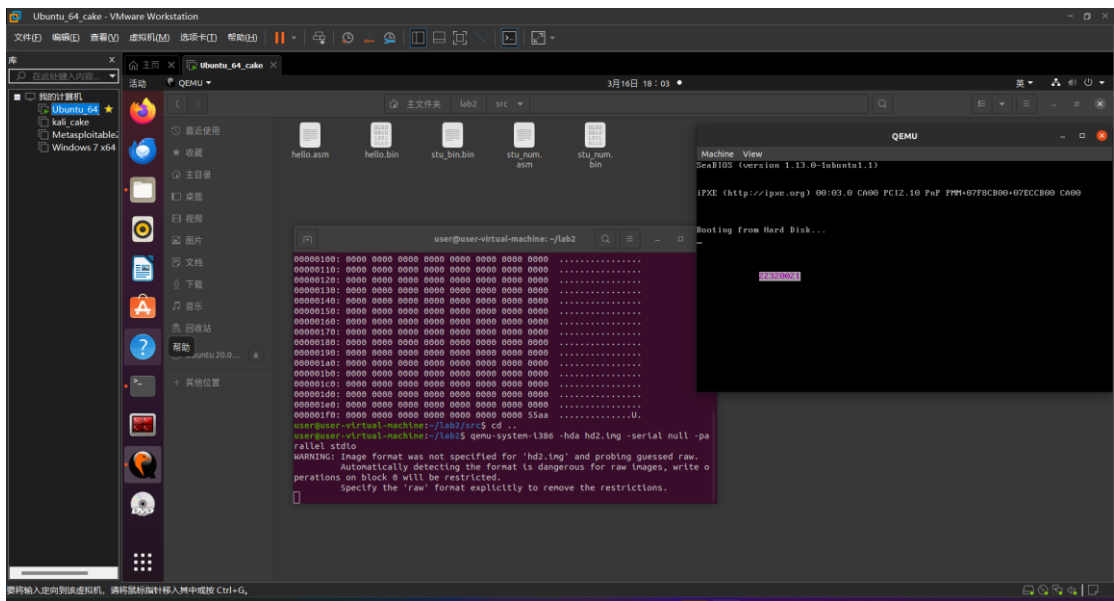
```
xor bh, bh
```

这是因为 `bx` 与 `ax` 都是 16 位的，而标号 `i`, `j` 的数据只有八位宽，为了保证结果正确性，采取赋值后高位取零的方法。

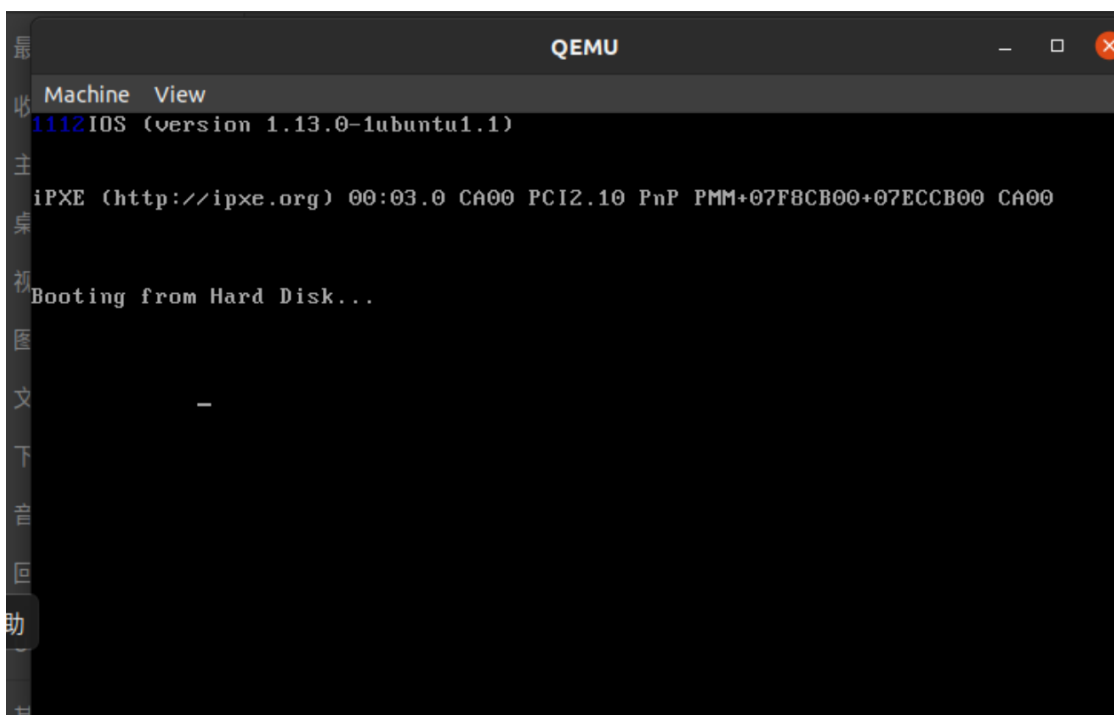
4. 实验结果



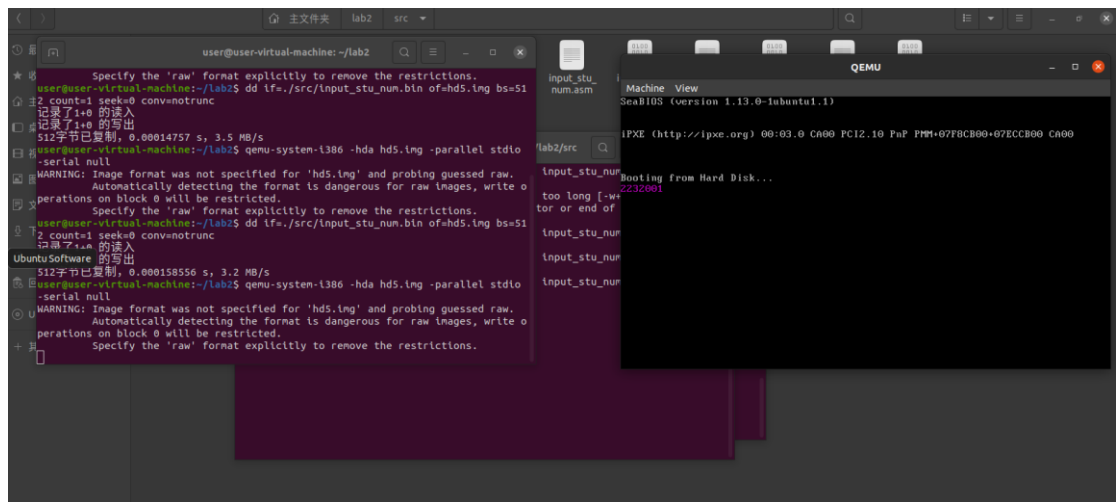
上图为复现 example1 的截图。



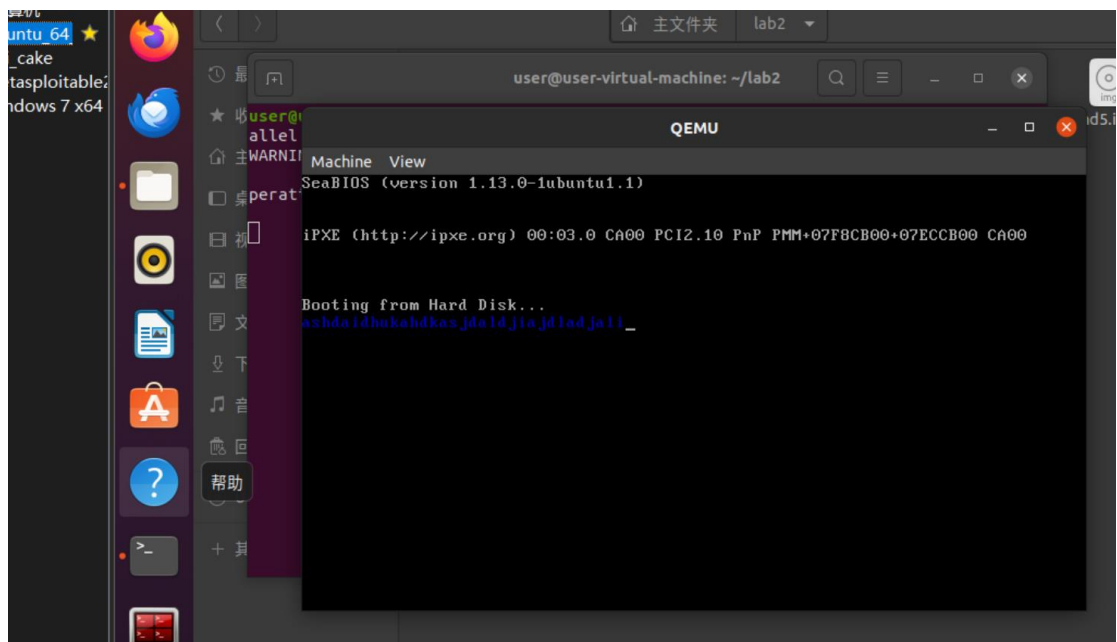
上图为输出自己学号的结果截图。



上图将光标设置到（11,12），并在（0,0）处显示光标的坐标每个坐标信息用两个字符表示。



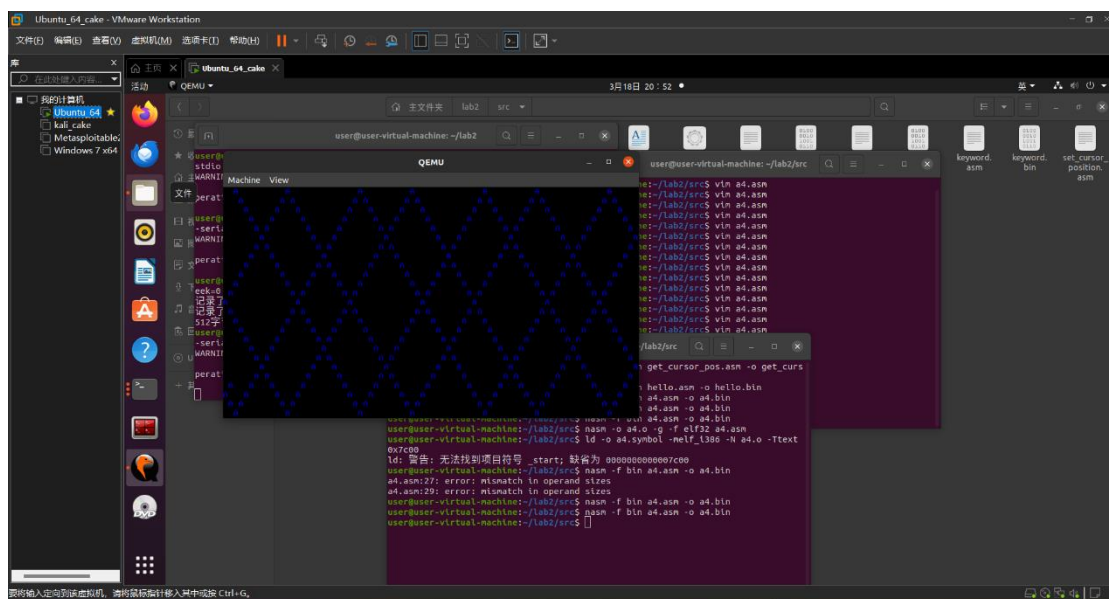
上图为采用实模式中断输出自己代码的结果图。



上述为中断实现键盘回显的效果截图。

```
user@user-virtual-machine: ~/lab2/assignment
user@user-virtual-machine:~/lab2/assignment$ vim student.asm
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ make run
>>> begin test
>>> if test pass!
>>> while test pass!
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$ ^C
user@user-virtual-machine:~/lab2/assignment$
```

上图为任务 3 实验截图。



上图为任务四结果截图。

5. 总结

通过这次实验，我学习了操作系统的启动过程以及汇编的相关知识。受益匪浅。实验内容中，建议可以再详细说明给定代码对寄存器的改动情况。（比如任务三

my_random 函数会修改 edx 寄存器。)